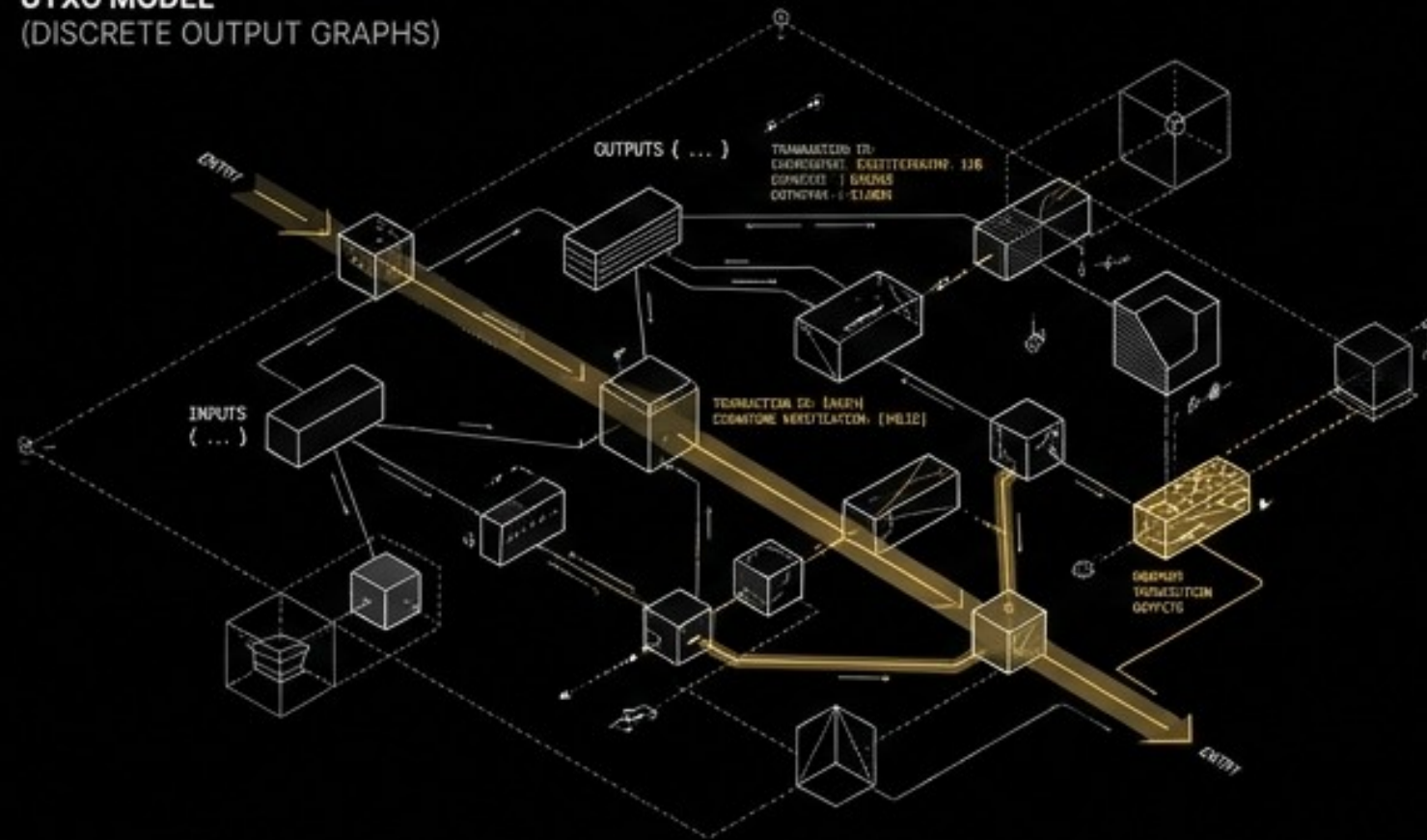


LEDGER STATE MODELS: UTXO VS. ACCOUNT MODEL

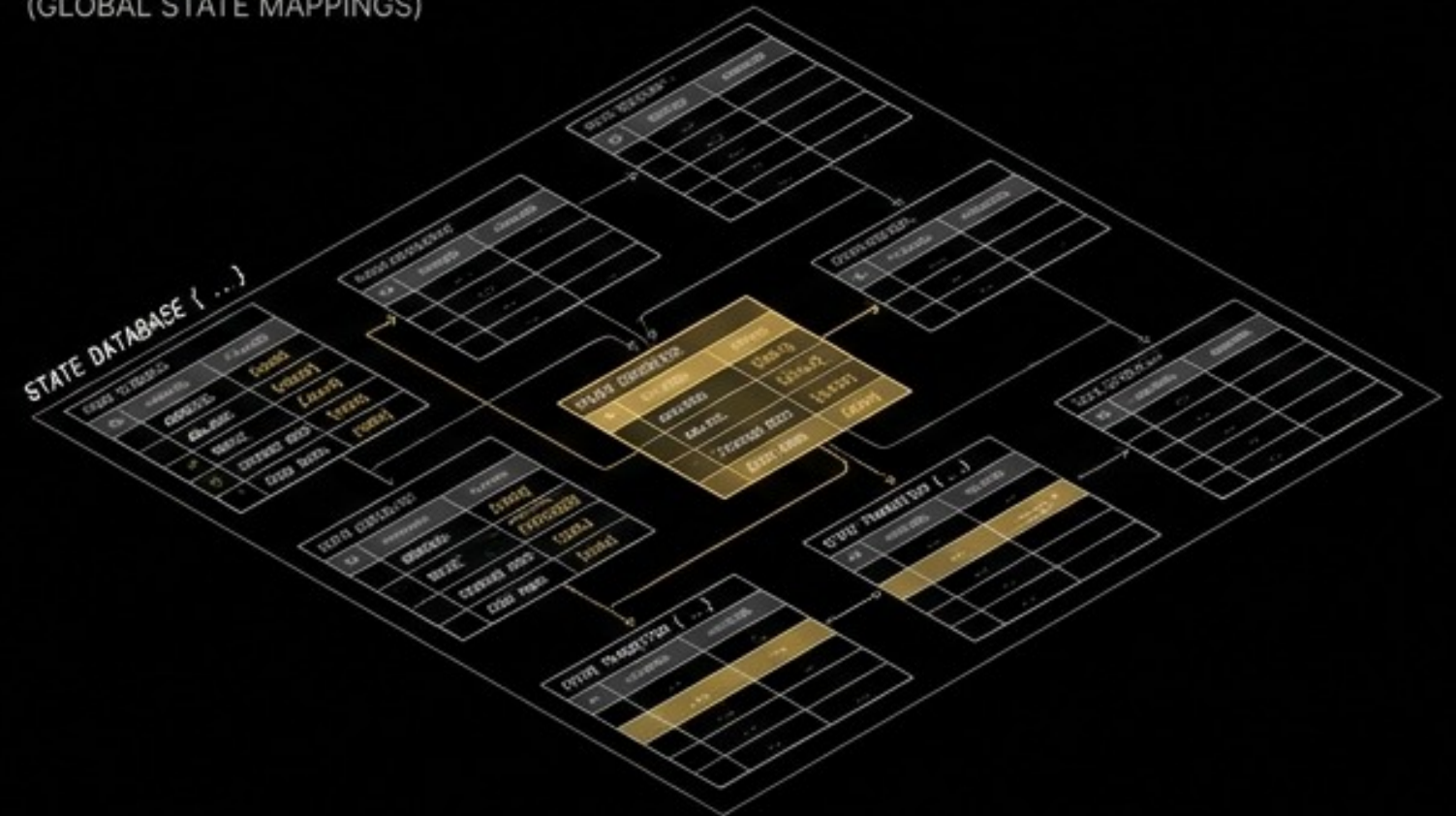
Architecture and State (02.3)

The architectural dichotomy between discrete output graphs and global state mappings dictates network **concurrency**, smart contract **expressivity**, and node hardware requirements.

UTXO MODEL
(DISCRETE OUTPUT GRAPHS)



ACCOUNT MODEL
(GLOBAL STATE MAPPINGS)



COMPARATIVE ANATOMY MATRIX:

<pre>{ "CONCURRENCY": { "UTXO": "PARALLEL VALIDATION", "ACCOUNT": "SEQUENTIAL EXECUTION" }, }</pre>	<pre>"EXPRESSIVITY": { "UTXO": "LIMITED (STATELESS)", "ACCOUNT": "TURING-COMPLETE (STATEFUL)" },</pre>	<pre>"SCALABILITY": { "UTXO": "HIGH (PRUNABILITY)", "ACCOUNT": "CHALLENGING (STATE GROWTH)" },</pre>	<pre>"SECURITY_SURFACE": { "UTXO": "NARROW (DOUBLE-SPEND PREVENTION)", "ACCOUNT": "BROAD (REENTRANCY, STATE BLOAT)" },</pre>
---	--	--	--

1

MODEL UTXO: PHYSICAL WALLET METAPHOR

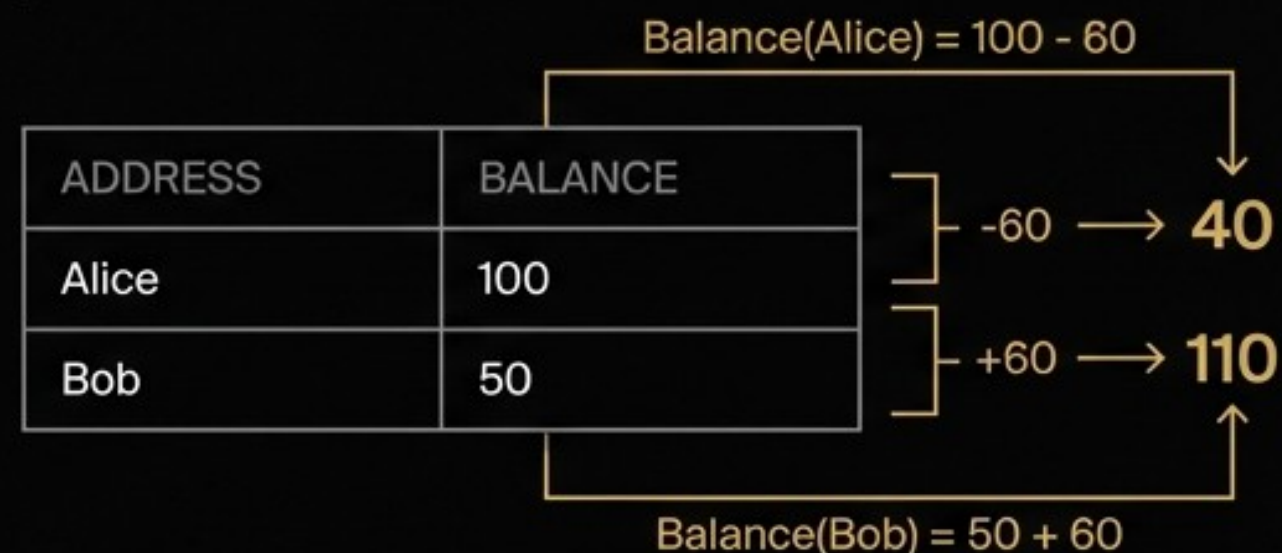
```
{ "Global Balance Entity": "Null",  
  "Structure": "Coins exist as discrete, indivisible chunks of  
value called Unspent Transaction Outputs (UTXOs).",  
  "Mechanism": "Buying a $60 item with a $50 and $20 bill  
destroys both bills, minting a new $60 bill for the merchant  
and a $10 change bill for the buyer."  
}
```



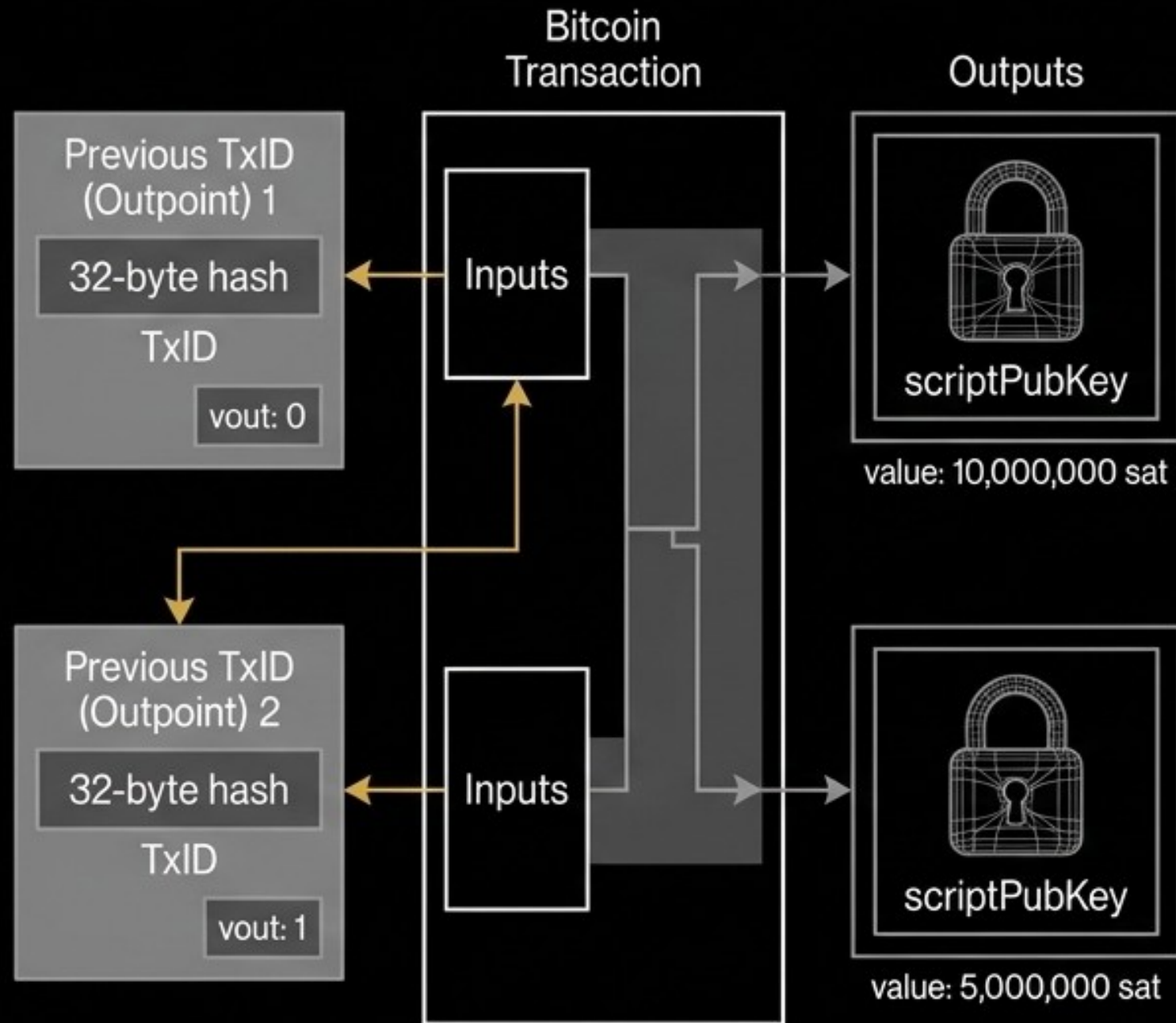
2

ACCOUNT MODEL: BANKING LEDGER METAPHOR

```
{ "Global Balance Entity": "Active",  
  "Structure": "A global database maps every address directly  
to its total balance.",  
  "Mechanism": "Value transfer executes via in-place  
arithmetic mutation of the database row:",  
  "Mutation": [  
    "Balance(Alice) = Balance(Alice) - 60",  
    "Balance(Bob) = Balance(Bob) + 60"  
  ]  
}
```



TRANSACTION ANATOMY



UTXO ANATOMY: POINTERS AND LOCKS

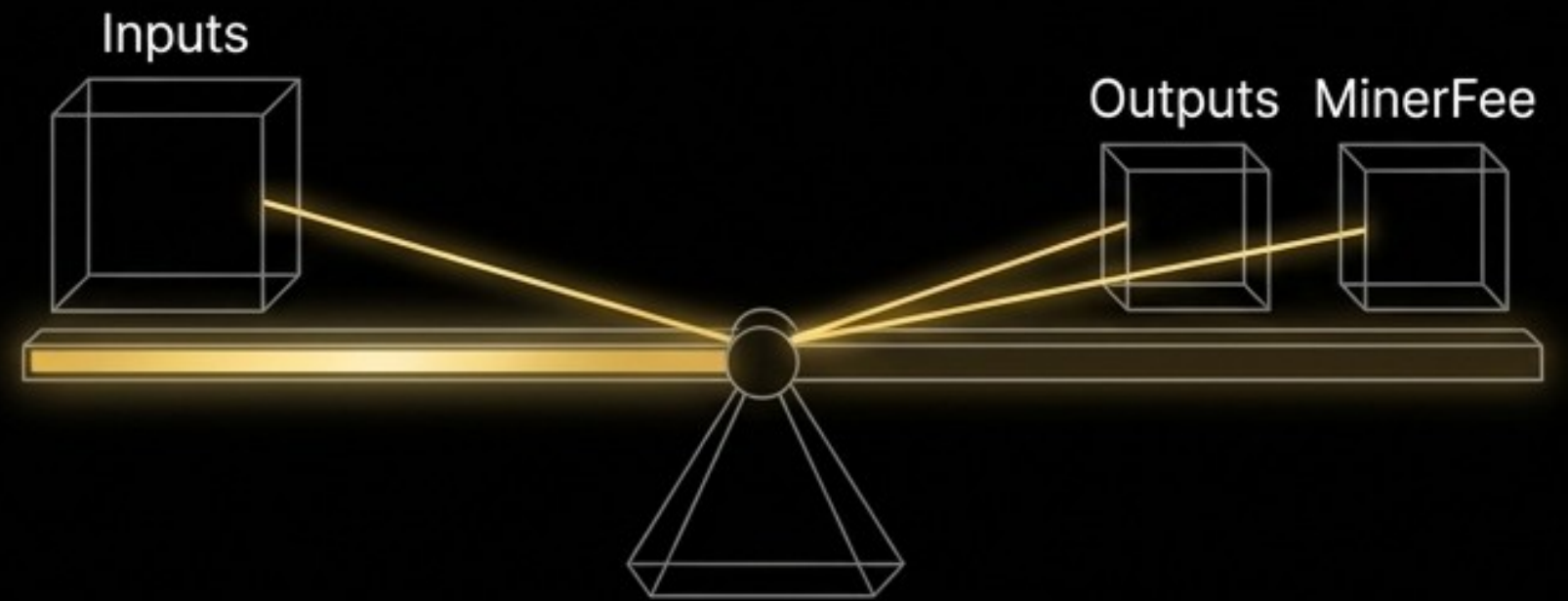
```
{  
  "UTXO Set": "The active database of all unspent  
transaction outputs since Genesis.",  
  "Transaction Inputs (Pointers)": {  
    "Inputs do not contain value. They point to past  
UTXOs using an Outpoint": {  
      "TxID": "32-byte hash of the past transaction.",  
      "vout": "4-byte integer index of the specific  
output.",  
      "scriptSig": "Cryptographic unlocking witness  
(signatures + public keys)."  
    }  
  },  
  "Transaction Outputs (Locks / TxOut)": {  
    "value": "Nominal satoshi amount.",  
    "scriptPubKey": "Cryptographic locking script  
dictating future spend conditions."  
  }  
}
```

Value Conservation and Implicit Fees

The Law of Value Conservation

$$\sum \text{Value(Inputs)} = \sum \text{Value(Outputs)} + \text{MinerFee}$$

```
{  
  Principle":  
  "UTXOs are indivisible. They must  
  be consumed 100% or not at all."  
}
```



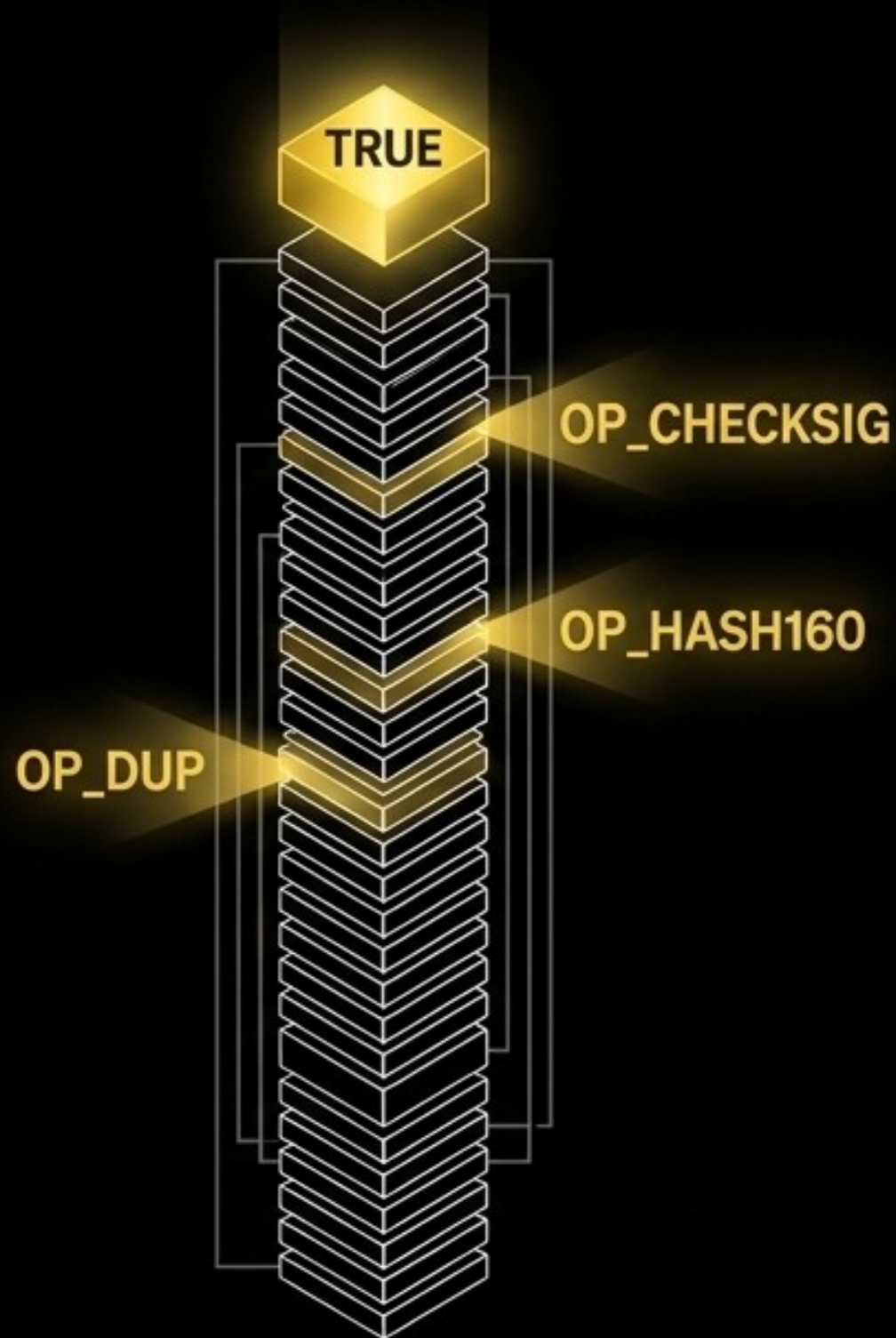
The Fatal Flaw of Implicit Fees

```
{  
  Miner fees are strictly implicit-calculated  
  purely as the mathematical difference between  
  inputs and outputs."  
}
```

```
{ The Danger":  
  If a wallet software forgets to create a  
  change output for the remaining funds, the  
  protocol instantly awards the entire excess  
  balance to the miner as a tip."  
}
```



CRYPTOGRAPHIC STACK EXECUTION



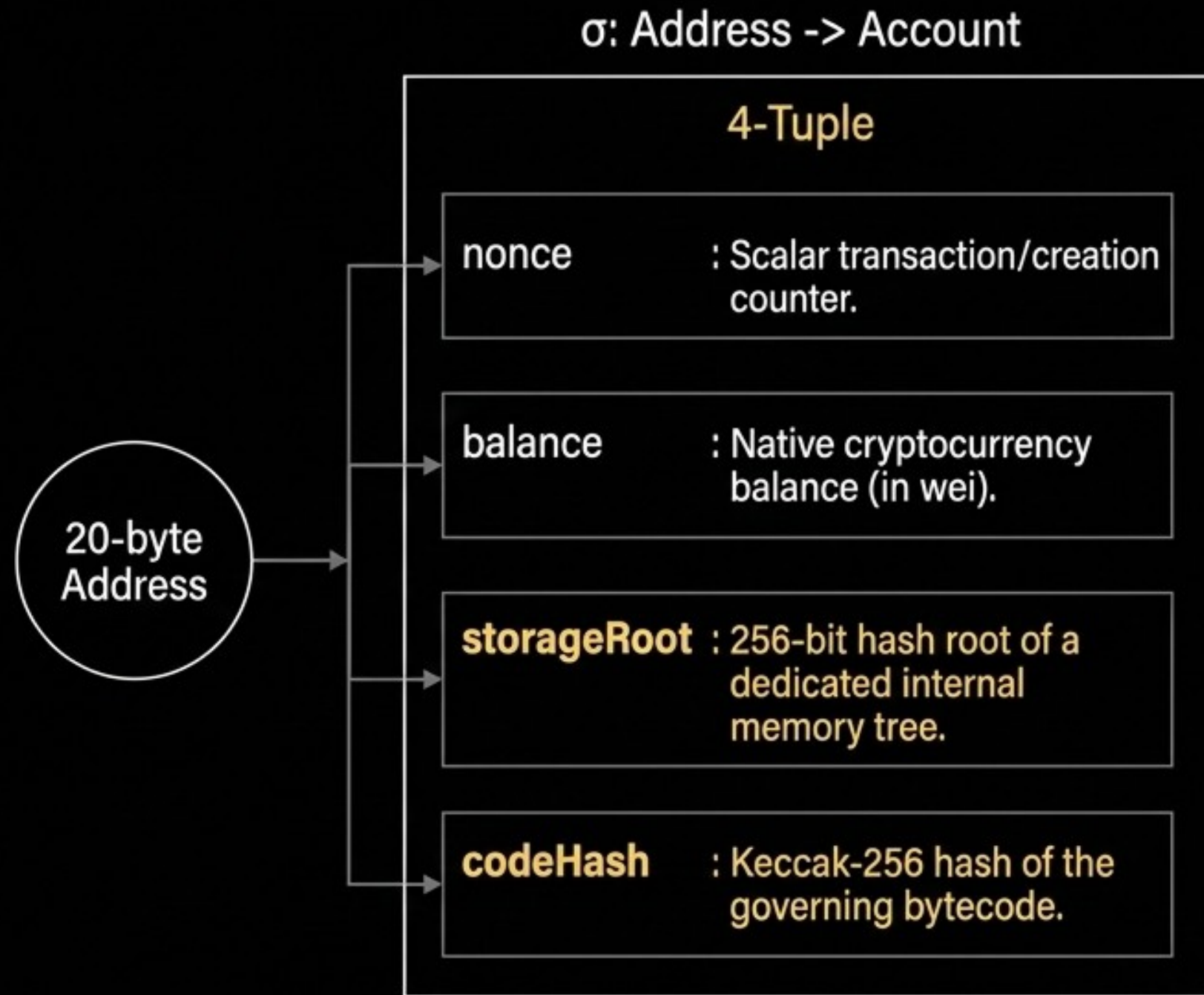
```
{  
  "Engine": "Forth-like, stack-based. Intentionally non-Turing  
complete (no loops) to prevent the halting problem."  
}
```

Execution Sequence (P2PKH):

1. Push signature **[sig]** and public key **[pubKey]** to stack.
2. **OP_DUP**: Duplicate the top stack item (**pubKey**).
3. **OP_HASH160**: Apply **SHA-256** and **RIPEMD-160** to **pubKey**.
4. Push target expected hash from **locking script**.
5. **OP_EQUALVERIFY**: Validate public key hash matches target address.
6. **OP_CHECKSIG**: Verify **ECDSA** signature against transaction digest.

Result: Spend is valid only if the final top stack value is **TRUE**.

Account Anatomy: The Ethereum 4-Tuple



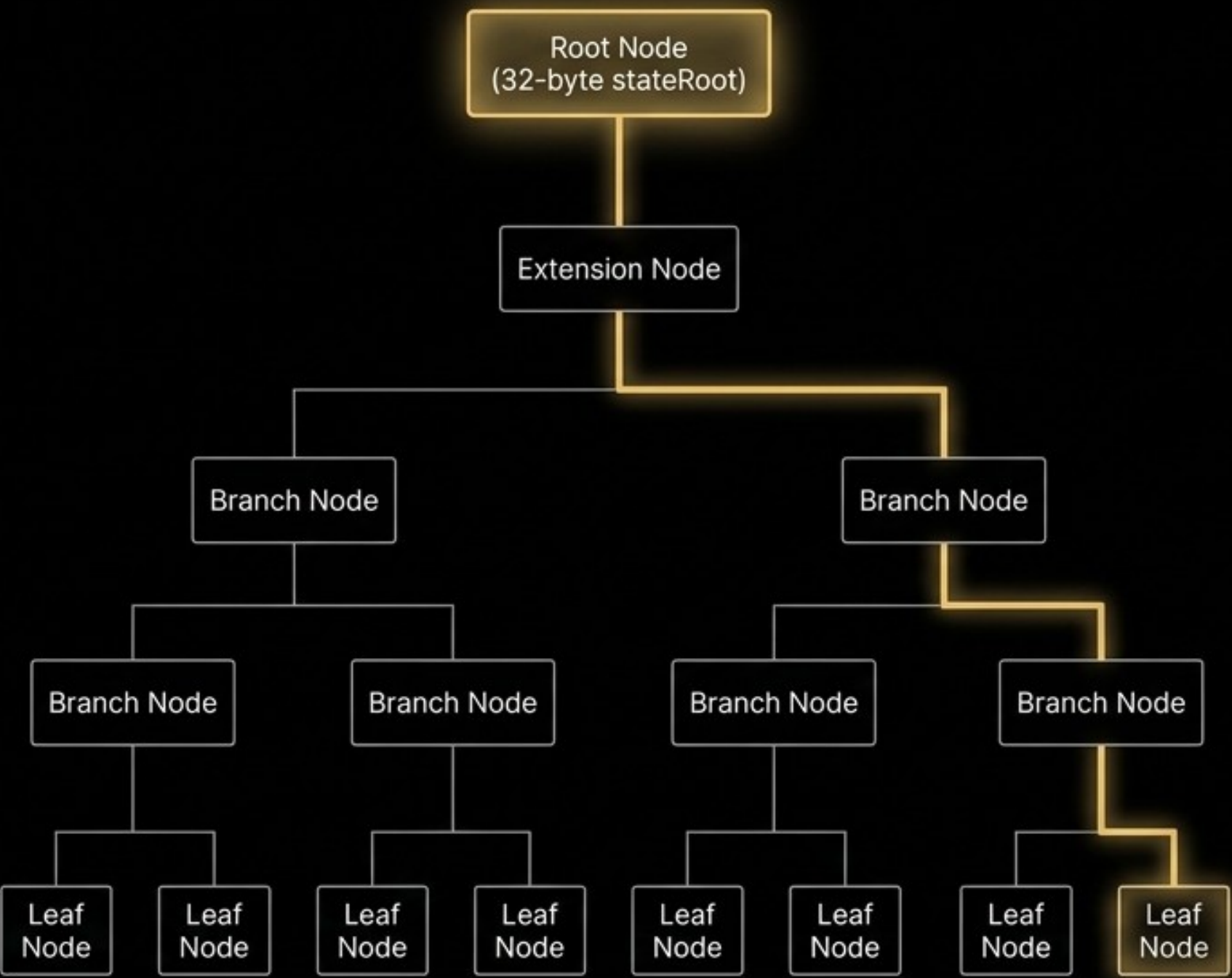
Externally Owned Accounts (EOA)

```
{  
  "Type": "Human/key-controlled",  
  "Attributes": {  
    "storageRoot": "Empty.",  
    "codeHash": "Empty."  
  },  
  "Role": "The only entities that can initiate transactions and pay gas."  
}
```

Contract Accounts

```
{  
  "Type": "Managed autonomously by permanent on-chain EVM bytecode.",  
  "Components": {  
    "nonce": "Scalar transaction/creation counter.",  
    "balance": "Native cryptocurrency balance (in wei).",  
    "storageRoot": {  
      "Description": "256-bit hash root of a dedicated internal memory tree."  
    },  
    "codeHash": {  
      "Description": "Keccak-256 hash of the governing bytecode."  
    }  
  }  
}
```

MODIFIED MERKLE PATRICIA TRIE (MPT)



```
{
  "The Hybrid Solution": {
    "Description": "Merges the cryptographic
integrity of a Merkle Tree with the O(log n)
path-finding efficiency of a Radix trie."
  }
}

{
  "Logarithmic Mutation": {
    "Mechanism": "When Alice transfers to Bob,
nodes do not recalculate the entire database.
Only the specific nodes along the traversal
path from leaf to root are updated."
  }
}

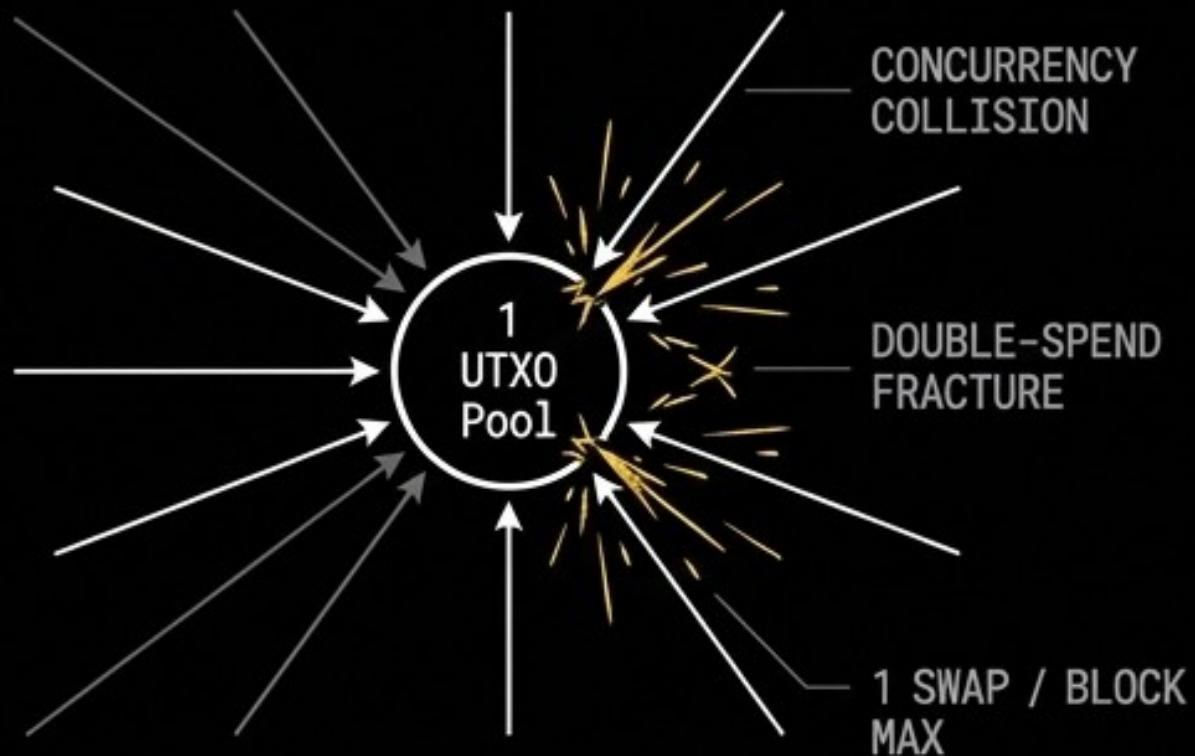
{
  "State Root": {
    "Outcome": "The resulting 32-byte stateRoot
is embedded directly into the block header,
guaranteeing global state consensus."
  }
}
```

Architectural Synthesis Matrix

Dimension	UTXO Model	Account Model
State Representation	Independent coin graph	Global mapping of balances
Transaction Concurrency	High (Native parallel processing)	Low (Forced sequential for same accounts)
Contract Expressivity	Limited (Stateless, complex shared state)	Turing-Complete (Effortless shared state)
Storage Burden	Efficient (Spent coins pruned from RAM)	High (Permanent state bloat)
Privacy Mechanics	Superior (New addresses for change)	Vulnerable (Static address reuse)
Double-Spend Check	Outpoint consumption check	Scalar nonce increment check

Real-World Architectural Bottlenecks

UTXO Limit: Cardano eUTXO



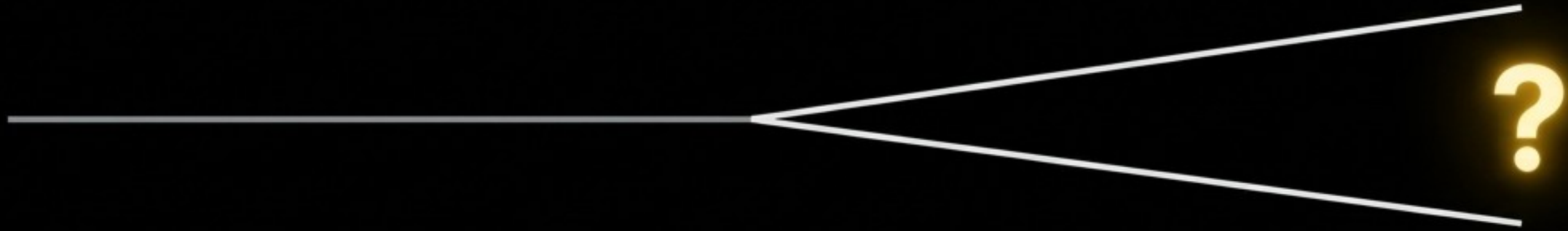
```
{  
  "Concurrency Collision": "Cardano adopted Extended UTXO for smart contracts. In an Automated Market Maker (AMM), liquidity is pooled."  
  
  "The Bottleneck": "Representing a liquidity pool as a single UTXO means it can mathematically only execute 1 swap per block. Simultaneous user swaps instantly failed due to double-spend collisions."  
}
```

Account Limit: Ethereum Storage



```
{  
  "State Bloat & Storage Contention": "Easy shared-state comes with severe hardware degradation. Sequential access to a shared liquidity pool triggers extreme gas bidding wars."  
  
  "The Bottleneck": "Account data and storage slots persist forever in the MPT. This continuous bloat chokes disk I/O, forcing full node validators onto enterprise-grade NVMe SSDs just to maintain chain sync."  
}
```

The Consensus Dilemma



Regardless of whether a ledger relies on the stateless graphs of UTXOs or the global mappings of Account models, all distributed nodes face one absolute mandate:

They must agree on a single, mathematically valid timeline.

But what happens when agreement fractures?
What occurs when two miners on opposite sides of the globe discover a valid block at the **exact same millisecond**?

How does a **network resolve temporary reorganizations versus permanent ideological hard forks**?

Next Module -> Forks, Finality, and Reorganizations